

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 1/23
---	--	-----------------------------

EDF - R&D

DEPARTMENT "MANAGEMENT OF INDUSTRIAL
RISKS"



April 2019

GROUP: RISK ANALYSIS
ON ORGANIZATIONS AND SYSTEMS

M. BOUISSOU

Knowledge Base Development Manual for KB3 v3 Software

HT-52/03 / 042 / J

Associated documents:

Document describing the syntax of the Figaro language; Figaro Language Reference Manual; Tutorial for using VisualFigaro HMI.

Summary:

The tool KB3 aims at automating dependability studies, thanks to the use of knowledge bases. It allows very large gains in the productivity of the studies of complex systems and allows users who are not dependability specialists to perform them. The knowledge bases consist of two parts, one relating to the semantics of the graphical models entered by the end user, described in Figaro language, and the other intended to parameterize the graphical interface. We give in this manual a procedure to develop a new knowledge base, in successive "layers", starting from the heart in Figaro language and gradually leading to a finalized and user-friendly application.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 2/23
---	--	-----------------------------

EXECUTIVE SUMMARY

The tool KB3 aims at automating dependability studies, thanks to the use of knowledge bases. It allows very large gains in the productivity of the studies of complex systems and allows users who are not dependability specialists to perform them. Knowledge bases are constituted by two parts, the first one relating to the semantics of the graphic models input by the end user, described in Figaro language, and the second one dedicated to the graphic interface customization. In this manual, we give a procedure to develop a new knowledge base, in successive "layers", starting from the heart of the knowledge base in Figaro language and gradually introducing a user-friendly application.

To minimize the development costs of tools around KB3, the procedure which we give is essentially based on the use of standard tools of the market, completed by the specific program VisualFigaro which provides help and sophisticated consistency checks of the information input by the user.

The stages of the development of a knowledge base (KB) are in summary:

- The construction of the "core" of the knowledge base
 - The construction in Figaro language of a first sketch of the knowledge base
 - The first tests of behavior of the models built thanks to a minimal interface
 - A first parameter setting of the KB3 GUI, intended to facilitate the graphic input of models by an automatic filling of the interfaces of Figaro objects from the links drawn by the user
 - The writing of the algorithms allowing to build deduced objects from manually input objects
- The improvements of the ease of use
 - Choose or create icons and styles of links
 - Exploit the inheritance in the file of definition of the GUI
 - Add display modes and graphic variants
- The "armor plating" of the application
 - How to protect the application against bad manipulations by the user
- The writing of the user's guide of the application and online help
- The maintenance of the KB, ensuring compatibility with existing studies

CONTENTS

1	Introduction	4
2	Constituent elements of a KB	4
3	The developer environment for KB3	5
3.1	The software	5
3.2	Configuring jEdit (important!)	5
3.3	Standard icons	6
4	Steps in the construction of a new KB	7
4.1	Create the starting files	7
4.2	Build the core of the model (in Figaro 1)	7
4.3	First tests in pure text mode	9
4.4	First tests with the KB3 graphical interface	10
4.4.1	Automatic generation of a first version of the .bdc file	10
4.4.2	Specification of the treatments associated with the KB	10
4.4.3	First tests	13
4.5	Specify connection points	13
4.6	Specify the connection constraints and the rules for filling the interfaces	14
4.7	Specify algorithms to build deduced objects	16
4.8	Choose or create icons and link styles	16
4.8.1	Icons	16
4.8.2	Links	17
4.9	Exploit inheritance in the .bdc file	17
4.10	Add display modes and graphic variants	17
4.11	Add display layers	18
4.12	Lock the application	18
4.13	Write help for the final user	19
5	Modification of an existing KB	19
5.1	General principles	19
5.2	How to reload the studies entered with a previous version of KB?	19
6	Conclusion	21
7	Appendix A: Passing parameters when KB3 launches an external code	22

Directory of document changes

Reference	Designation of changes	Comments
A	Officially released initial version	
B, C, D, E ...	Updated versions to take into account developments in KB3 and knowledge base editing tools. No official broadcast.	

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 4/23
---------------------------	--	-----------------------------

1 Introduction

The purpose of the KB3 tool is to automate dependability studies through the use of knowledge bases (KB). It allows very large gains in the productivity of complex systems studies and allows non-specialists in dependability to perform them. The knowledge bases consist essentially of two parts, one relating to the semantics of graphical models entered by the end user, described in Figaro language, and the other intended to parameterize the graphical interface. We give in this manual a procedure to develop a new knowledge base, in successive "layers", starting from the heart in Figaro language and gradually leading to a finalized and user-friendly application.

This manual is for developers who are already familiar with the concepts found in a knowledge base for KB3. It is to be taken as a kind of practical check-list intended to save time. It is therefore deliberately succinct.

In the first part, we will describe the constituent elements of a knowledge base, and the set of tools that we call the "developer environment for KB3".

Then we will give a procedure which, followed step by step, allows the progressive construction of a knowledge base by exploiting these tools. We can make an analogy between this procedure and the notion of paint "layers", to move from a sketch made in a few hours to a complete and very friendly application, with a simple online help and a documentation.

Finally, we will show that the modification of an existing KB can also be based on this procedure.

Note: There are two documents that serve as a reference on the Figaro language. The "Figaro language reference manual" is a classic document that can be read sequentially; it contains many examples. The document "Syntax of the Figaro language" is an exhaustive definition of the syntax by syntactic diagrams, as concise as possible to allow a quick navigation.

2 Constituent elements of a KB

A knowledge base consists of the following files:

- Base.fi: description of classes in Figaro language;
- Base.bdc: parameterization of the graphical interface, algorithms calculating the deduced objects, and definition of menus specific to the KB. This file is in XML syntax, and must follow a well-defined schema. This schema is described in the file Bdcfr.xsd or Bdceng.xsd depending on the language of the KB (French or English);
- For each class of the KB having a graphic representation:
 - Icon_class_i.ico: icon in bitmap (format 24x24 is **imperative**) destined to appear in the palette in the left pane of the KB3 interface; this is why there must be one and only one for each Figaro class that we want to appear in this palette,
 - Icon_class_i.sym: icon in a proprietary format to the graphic library Stingray which was used as a basis for the development of KB3 v3. Possibly, several .sym icons can be associated to a Figaro class: one or the other of these graphic variants will be displayed on a system diagram according to conditions expressed in Figaro language.
- One or more help files (.doc or .pdf).

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 5/23
---------------------------	--	-----------------------------

3 The developer environment for KB3

3.1 The software

The procedure that we will give in this document is based on a collection of free tools that have been chosen for their effectiveness in different functions that are necessary for the construction of the KB. These tools all exist in "portable" version, so they can be installed without administrative rights (except for Java).

The "minimal" environment consists of (in the order of the installations to be done):

- The runtime environment of Java,
- The open source text editor jEdit and additions to jEdit grouped in an installer called "VisualFigaro".

Complementary software (easy to find on the Internet, except Designer which is systematically delivered with KB3) to work on KB3 knowledge bases:

- Designer to edit icons in .sym format. Designer is the **only** tool for editing icons in this format, unique to the Stingray graphic library. It is always delivered with KB3 (it is the executable Designer.exe present in the installation directory of KB3).
- IcoFX, AWicons lite etc. to edit icons in .ico format. These tools offer a simultaneous display mode of the enlarged icon (for editing) and in final size (so that we see in real time the effect produced); they allow the use of a palette reduced to 16 colors, drawing geometric shapes (circles ...) and text.
- QxmlEdit is a simple editor for XML files. It has the advantage of being lightweight to install and rich in functions. It clearly shows the tree structure of the XML file and separates the "tags" from the contents of the fields. It checks the file for compliance with the associated schema, which can be useful for looking for errors in the .bdc file. It also makes it possible to graphically display as a tree schemas such as bdcfr.xsd and bdceng.xsd. Another very light tool is good at this task: XSD Diagram.

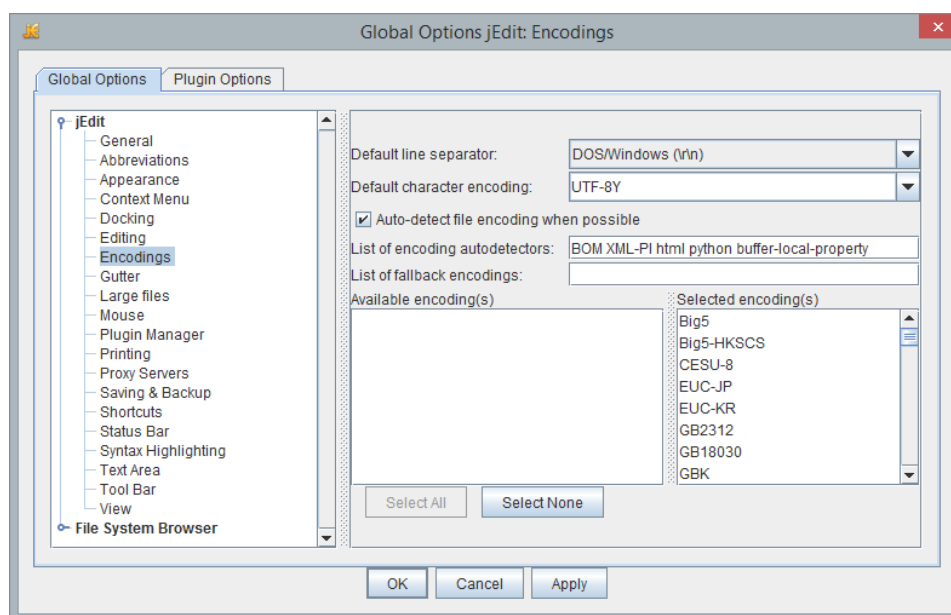
The first work to be done is obviously the installation of the developer environment. The installation of all the tools mentioned above is very fast. The main installer (jEdit + VisualFigaro) which contains a "portable" version, i.e. fully autonomous and that does not require administrative rights is available on Sourceforge, at: <http://sourceforge.net/projects/visualfigaro/>.

3.2 Configuring jEdit (important!)

It is important to choose some jEdit options, for a comfortable use of VisualFigaro.

To prevent accented letters from being transformed into illegible characters, it must be checked that in "Utilities / Global options / jEdit / Encodings" (see screen shot below) the default encoding option is UTF8-Y or UTF-8 (in principle this is done by the installation, but there may be interference created by a jEdit setup already installed on the machine).

In addition, in the "Gutter" section of this same options panel, check the "line numbers" box because the display of line numbers will be useful to quickly find the syntax errors reported by the knowledge base checker.



In the "Editing" section, you must take the "indent" option for "folding mode", and take the "Initially collapse folds with level 1" option which allows you to have an initial view of a knowledge base hiding the details. Just click on the triangle in front of a set of hidden lines to reveal an additional level of detail (shift + click if you want to "unfold everything" at one time). This function of jEdit is particularly useful for editing the .bdc file (provided you maintain its indentations).

You can choose the language of the jEdit menus in the "Utilities / Global options / jEdit / General" section. The HMI of the VisualFigaro plugin will stay in English.

3.3 Standard icons

To avoid endless duplication of icons, this "basic" icon library is provided to the KB3 developer. Icon names must respect the following rules:

Icons have English names if they are bilingual; if they are not, the French version bears the same name as the English version, with a suffix _f, so that they follow each other in alphabetical order.

Example: and_gate / and_gate_f

The name of an icon should be as descriptive as possible, without reference to a particular context: "white_circle" rather than "state" is used to designate an icon that can represent a state in a Markov graph. It must not contain a blank " ".

As long as an icon only exists in one color, that color is not specified in the name. Same for the size.

There are variants by simple colors: white, black, yellow, blue, red, green, etc ...

We will have at most three sizes: small, med (ium), big.

The medium size will correspond to an image inscribed in a square of 48x48 pixels (twice the size of .ico icons). Will be considered as "small" any size below this limit, and as "big" any size larger. This size qualifier only applies to the vector icon: the same .ico file can be associated with three different vector icons.

A name will give, in this order (and separated by "_"):

- the size (obligatory),
- the color (optional),
- the geometric form (obligatory),
- the possible suffix _f for the language.

Examples: med_green_square, big_red_circle, med_and_gate, med_and_gate_f

Link icons: for links, there is no .sym file.

The name of a link .ico file will necessarily include the following elements, separated by "_":

- type of line (solid, dash, dotted) (in the .bdc file: SOLID, DASH, DOTTED)
- color (same conventions as for node icons)
- type of ends, in the form of a short name, taken in the following table (the last two lines correspond to lines with an arrow in the middle):

LINE_LINE	ll
LINE_THREE_POINTS	l3p
LINE_FOUR_POINTS	l4p
THREE_POINTS_LINE	3pl
THREE_POINTS_THREE_POINTS	3p3p
THREE_POINTS_FOUR_POINTS	3p4p
FOUR_POINTS_LINE	4 seats
FOUR_POINTS_THREE_POINTS	3p4p
FOUR_POINTS_FOUR_POINTS	4p4p
LINE_THREE_POINTS_LINE	l3pl
LINE_FOUR_POINTS_LINE	l4pl

- thickness (integer: 1, 2, ...)

Examples: solid_green_ll_2, dash_yellow_3p3p_1 ...

4 Steps in the construction of a new KB

Preamble: this document complements (on "substantive" issues) information limited to the practical aspects of the use of the VisualFigaro GUI (with many screenshots) available in the document entitled "VisualFigaro Tutorial".

4.1 Create the starting files

Choose a development directory and create the necessary files from VisualFigaro, with the command: "KB management / New KB".

This creates a directory named "KBname", containing:

- an "icons" directory, containing a minimal set of "standard" icons (including simple geometric shapes),
- the Bdcfr.xsd or Bdceng.xsd schema file, depending on the language chosen for the KB (French or English),
- a file "KBname.fi" initialized by a few lines taken from file "copiedFiles.xml" of environment VisualFigaro.

4.2 Build the core of the model (in Figaro 1)

The core of the knowledge base is a text in Figaro language of order 1: you can use the syntax help available by a right click on a tree element on the left pane.

It is important to check the validity of the KB often by clicking on the large vertical button.

Suppose we have entered the following knowledge base:

```
(* Small pedagogical knowledge base *)
(* It allows to describe meshed telecommunication networks
   with failures on the nodes and the links.
```

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 8/23
---	--	-----------------------------

It has two possible uses: simulation (with the group of rules simu_group) and generation of fault trees (with group of rules FT_group) *)

(* Author: M. Bouissou, 20 dec 2007 *)

GROUP_NAMES

simu_group ;

FT_group ;

CLASS node GR_NODE ;

CONSTANT

function DOMAIN 'source' 'target' 'intermediate' DEFAULT 'intermediate' ;

lambda DOMAIN REAL DEFAULT 1e-5 ROLE DESIGN;

mu DOMAIN REAL DEFAULT 0.1 ROLE DESIGN;

FAILURE fail LABEL "Failure of %OBJECT"

RELIABILITY_DATA

GROUP FT_group

MODEL_GLM

GAMMA 0.

LAMBDA lambda

MU mu ;

EFFECT connected LABEL "%OBJECT is linked to a source" ;

OCCURRENCE

GROUP simu_group

MAY_OCCUR

FAULT fail

DIST EXP(lambda);

GROUP simu_group

MAY_OCCUR

REPAIR rep REPAIRS fail

DIST EXP(mu);

INTERACTION

rule1

IF WORKING AND function = 'source'

THEN connected;

CLASS source KIND_OF node ;

CONSTANT

function DEFAULT 'source';

CLASS target KIND_OF node ;

CONSTANT

function DEFAULT 'target';

CLASS link GR_LINK;

CONSTANT

link_lambda DOMAIN REAL DEFAULT 1e-5 ROLE DESIGN ;

link_mu DOMAIN REAL DEFAULT 1 ROLE DESIGN ;

FAILURE

interruption LABEL "The link %OBJECT is broken"

RELIABILITY_DATA

GROUP FT_group

MODEL_GLM

GAMMA 0.

LAMBDA link_lambda

MU link_mu ;

OCCURRENCE

GROUP simu_group

MAY_OCCUR

FAULT interruption

DIST EXP(link_lambda);

GROUP simu_group

MAY_OCCUR

REPAIR rep REPAIRS interruption

DIST EXP(link_lambda);

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 9/23
---------------------------------------	--	-----------------------------

```

CLASS mono_dir_link KIND_OF link ;
INTERFACE
    start KIND node CARDINAL 1 EDITION NOT MODIFIABLE;
    end KIND node CARDINAL 1 EDITION NOT MODIFIABLE;
INTERACTION
    rule1
    IF WORKING AND connected(start) AND WORKING(end)
    THEN connected(end) ;

CLASS bi_dir_link KIND_OF link ;
INTERFACE
    extremity KIND node CARDINAL 2 EDITION NOT MODIFIABLE;
INTERACTION
    rule1
    IF WORKING AND
    (FOR_ANY x AN extremity WE_HAVE WORKING(x)) AND
    IT EXISTS x AN extremity SUCH THAT connected OF x
    THEN FOR_ALL z AN extremity DO connected(z);

CLASS fail_counter ;
(* Count of failures in the system (nodes and links) *)
ATTRIBUTE
    nb_failures DOMAIN INTEGER DEFAULT 0;
INTERACTION
    rule1
    GROUP simu_group
    THEN nb_failures <-- SUM FOR_ALL x AN OBJECT OF_CLASS node OF_TERMS (1 * fail(x)) +
    SUM FOR_ALL y AN OBJECT OF_CLASS link OF_TERMS (1 * interruption(y)) ;

(* A SYSTEM_OBJECT is a global object that can access the variables of all objects
    forming part of the system *)
SYSTEM_OBJECT Failure_counter IS_A fail_counter ;
(* ***** END ***** *)

```

4.3 First tests in pure text mode

To test the knowledge base during construction, it is necessary to be able to quickly define small "systems" that is to say sets of objects whose interfaces, values of constant characteristics (such as failure rates) etc. can be chosen.

It is possible to do this (since v2.00 of VisualFigaro) while remaining in purely textual mode, so without wasting time to configure the GUI of KB3. This possibility is interesting because it often happens that some choices made in the definition of classes are called into question, and in this case the updating of the information relating to graphic aspects can waste time.

Using VisualFigaro's "Object Management" menu, you can create a model, that is, a file containing object descriptions. The "New Object" command launches a small wizard that makes it easy to create an object from the choice of the class to instantiate and some information entered in a dialog. At the end of the dialog, the object is written in the model file, in Figaro syntax. You can obviously also write directly in the file!

Once the file containing the model is complete, you can run various commands on it:

- Check (Check Model);
- Instantiation in Figaro 0 (Generate Figaro 0);
- The launch of the interactive debugger (Run Fig0debug);
- Generation of a fault tree (Generate Fault Tree);
- The launch of the Figseq and YAMS calculation tools¹.

Each of these treatments (apart from checking) begins by presenting a window of choice of options. The choices to make are easy to understand, with the possible exception of the choice of rule groups to instantiate. In

¹ For Figseq and YAMS, it is necessary to install these codes separately from Visual Figaro, and to refer to the indications of §4.4.2 to possibly inform by hand the file KB3Configuration.xml.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 10/23
---	--	------------------------------

particular, think about selecting the group WITHOUT_NAME (SANS_NOM in French) if this is relevant. If no group is selected in the list, the rules of *all* groups, including the group WITHOUT_NAME, are instantiated.

The proposed options do not include all the possibilities that exist from the KB3 GUI, but only the most common ones, sufficient at the beginning of the knowledge base development.

4.4 First tests with the KB3 graphical interface

Working in purely textual mode is laborious if you have to define many objects with their interfaces. It becomes difficult to have a global and synthetic view of the manipulated model. If so, it's time to move to using KB3 to graphically enter the models.

To do this, you must load the knowledge base in KB3, which is only possible with a .bdc file containing a minimum of information.

4.4.1 Automatic generation of a first version of the .bdc file

Using the "KB Management / Class Hierarchy" menu of the VisualFigaro plugin, generate a representation of the class hierarchy as text. In our example, we obtain this:

```
-FIGARO
  | -node
  |   | -source
  |   | -target
  | -link
  |   | -mono_dir_link
  |   | -bi_dir_link
  | -fail_counter
```

Then, add key words GR_LINK or GR_NODE just after the names of the types (with the highest possible level in the hierarchy) to differentiate the links of the nodes.

In our example, this leads us to modify two lines of the knowledge base:

```
CLASS node GR_NODE;
CLASS link GR_LINK;
```

Generate the .bdc file from the .fi file by the command: "Edit XML File" of the plugin, taking the option of generating data by default (the option "Start from scratch" is not recommended for non-experts because it requires to know the minimum data you need to enter in order to load the knowledge base into KB3). This operation is to be done only once for a given knowledge base (it is proposed by VisualFigaro only if the .bdc file is missing when one starts the editing command of this file). Subsequently, if you add classes in the .fi file, the fastest is usually to copy and paste from classes already declared in the .bdc file using QxmlEdit (or jEdit) to add them too.

Anyway, from this point, it is advisable to periodically open the .bdc file in QxmlEdit. Thus we have the possible error messages due to a non-compliance to the schema before attempting to load the knowledge base in KB3. Most of these messages are due to incompletely documented structures.

The .bdc file generated does not contain the intermediate classes (which are not leaves in the Figaro class hierarchy): it will be possible to add some to factorize information (see § 4.9) common to several terminal classes.

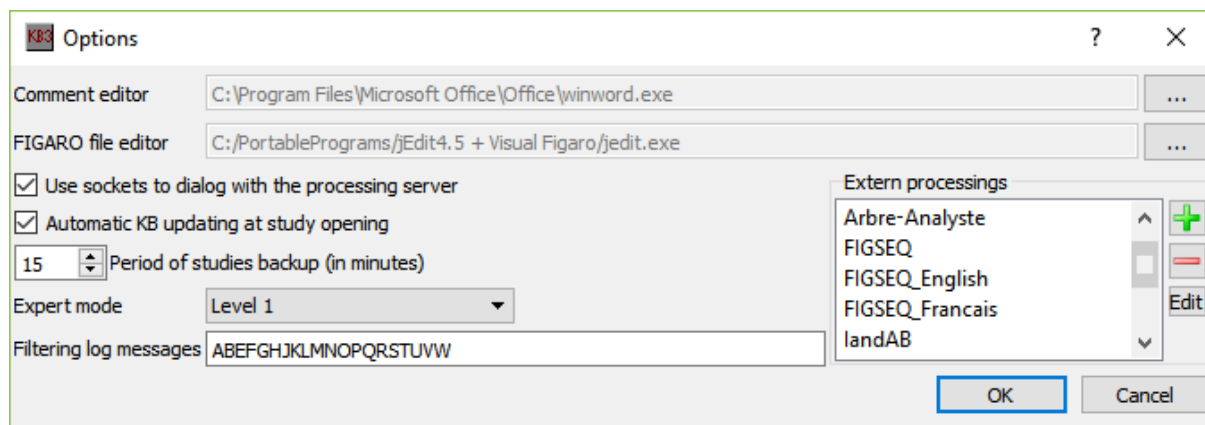
Each **node** terminal class is provided with standard "ports" (points designated by their coordinates in X and Y in the rectangle by which the node icon is circumscribed) "in" and "out" to which all links have the authorization to hook.

The **link** terminal classes all have the same characteristics: they are black, in solid line of thickness 1, ended by an arrow and they are authorized to go from a port "out" to a port "in".

4.4.2 Specification of the treatments associated with the KB

With regard to the treatments associated with the KB, VisualFigaro generates default data (to be possibly rectified, especially as regards the groups of rules) for the three types of treatments: generation of fault tree, interactive simulation, instantiation in Figaro 0. These processing "models" (sort of templates) can be modified, deleted if they have no meaning for the KB under construction, or duplicated and adapted so to offer the user several "ready to use" models.

In addition, it is possible to add external code launches that allow to automatically chain operations such as the generation of a fault tree and/or a model in Figaro 0 and the launch of an external code (for example: RiskSpectrum or YAMS). To configure or check the list of codes that can be called from KB3, it is necessary to use the Tools / Options panel (with expert level 1 or more) which offers an HMI to fill the file KB3Configuration.xml² (this function only exists since 2018: earlier, it was necessary to edit the file directly, outside of KB3):



The screen shot above shows a list of codes declared in the KB3Configuration.xml file. For details of the launch configuration of a particular code, cf. Appendix A.

Of course, you also need to install the computer codes you intend to use. For Figseq, YAMS and GRIF.Tree calculation codes, there are installers that automatically declare these codes in the KB3Configuration.xml file³. Here is an example of the file KB3Configuration.xml (the first fields of the file are used to memorize the locations of the last data used by KB3. All tags and keywords are in French, even for the English version of KB3):

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<configuration_KB3 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
fichier_bdi="E:\KB3Data\db\DigDyn.mdb" fichier_bdci="E:\KB3Data\2 BDC\BDC
DigDyn\V1.0\1_Sources\English\DynDig.bdc"
xsi:noNamespaceSchemaLocation="KB3Configuration.xsd">
  <code>
    <nom>FIGSEQ</nom>
    <chemin_exe/>
    <chemin_IHM>C:\Program Files\EDF MRI Tools\FIGSEQ\figseq-gui.exe</chemin_IHM>
    <chemin_traducteur/>
    <chemin_pretraitement/>
    <chemin_posttraitement/>
    <sous_repertoire>Figaro0</sous_repertoire>
    <ADD_requis>FAUX</ADD_requis>
    <Figaro0_requis>VRAI</Figaro0_requis>
    <base_de_faits_requise>FAUX</base_de_faits_requise>
  </code>
```

² Note that VisualFigaro also uses this file to find out where YAMS and Figseq are.

³ In fact, there may be two KB3Configuration.xml files: a file located in C:\ProgramData\EDF MRI TOOLS, whose data is accessible to all users of the machine, and which is not modified by KB3, and a user-specific file, located in the directory: C:\Users\username\AppData\Roaming\EDF MRI TOOLS. At each launch, KB3 reads the two files and takes the union of the information it finds there. In case of conflict for a given code, it takes the information of the file specific to the user.

To avoid conflicts, only the file specific to the user is modified by the installers of the calculation codes and by KB3. This file is rewritten by KB3 at the end of each session as a single line. To read it conveniently, you need to reformat it with an XML editor.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 12/23
---------------------------------------	--	------------------------------

```

<code>
  <nom>YAMS</nom>
  <chemin_exe/>
  <chemin_IHM>c:\Program Files\EDF MRI Tools\YAMS\yams-gui.exe</chemin_IHM>
  <chemin_traducteur/>
  <chemin_pretraitement/>
  <chemin_posttraitement/>
  <sous_repertoire>Figaro0</sous_repertoire>
  <ADD_requis>FAUX</ADD_requis>
  <Figaro0_requis>VRAI</Figaro0_requis>
  <base_de_faits_requise>FAUX</base_de_faits_requise>
</code>
</configuration_KB3>

```

If a knowledge base contains calls to codes not declared in KB3Configuration.xml, this file must be completed directly or using the KB3 options panel.

Note: In most cases (and as in the examples above), the only useful <chemin_x> field is <chemin_IHM>. If other fields must be filled in, you must be careful that for the pre and post processing (tags <chemin_pretraitement> and <chemin_posttraitement>), you must give the name of the **directory** containing them, because the names of the executable files of these programs (namely "pretraitement.bat" and "posttraitement.bat") are "hard coded" in KB3.

The <sous_repertoire> field gives the name of a subdirectory of the directory KB3data/extern (KB3data designating the directory which contains the database loaded in KB3) in which will be created the temporary data necessary to launch the corresponding external code.

The preliminary operations described above being done, it remains now to insert into the .bdc file of the knowledge base being built (just after the Figaro 0 generation model) one (or more) model (s) for launching external code (s).

The only link between the declaration of an external code in the .bdc file and the KB3Configuration.xml file is the name of the code. It is therefore necessary to be careful not to input a wrong name in the VisualFigaro HMI (there is no control).

Here is an example of what is introduced in the .bdc file following the declaration of an external processing to launch Figseq without having to manually generate the Figaro 0 beforehand.

```

<EXTERNAL_PROCESSING_MODEL>
  <NAME>FIGSEQ</NAME>
  <CODES>
    <ALL_CODES>FALSE</ALL_CODES>
    <CODE_NAME>FIGSEQ</CODE_NAME>
  </CODES>
  <FT>FALSE</FT>
  <FIG0>TRUE</FIG0>
</EXTERNAL_PROCESSING_MODEL>

```

Note that there is redundancy of information between the fields:

```

<FT>FALSE</FT>
<FIG0>TRUE</FIG0>

```

of the .bdc file and the fields:

```

<ADD_requis>FAUX</ADD_requis>
<Figaro0_requis>VRAI</Figaro0_requis>

```

of the KB3Configuration.xml file.

The information in the .bdc file is taken into account when displaying the KB3 window "Launch external processing" to gray out some boxes, but when the user clicks to actually start a processing, a check of compliance with what is mentioned in the KB3Configuration.xml file is made and possibly triggers an error message. It is therefore necessary to make sure that the .bdc file complies with what is specified for the code to be launched in the KB3Configuration.xml file.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 13/23
---	--	------------------------------

If groups of rules have been defined in the .fi file, it is necessary to specify in the .bdc file the groups of rules to be considered for each type of treatment. Account must be taken of the fact that rules not assigned to any group are considered part of the group WITHOUT_NAME. If you do not specify which rule groups to use for a given treatment, *all* the rules are used. The rule group choices made in the knowledge base can be declared as editable or not by the knowledge base user in KB3.

4.4.3 First tests

At this stage, you can load the KB into KB3 and test by creating objects. All objects (nodes and links) can be created, and you can connect any link on any node, but it does not affect the interfaces: you have to fill them "by hand", from the object editor. The icons are all the same as the default icon.

After entering a few objects, you can start treatments: you must be careful with the choice of groups of rules, for each type of treatment. Depending on the results, you can delete or modify the treatment models.

4.5 Specify connection points

The KB3 HMI allows you to define "smart" graphical interfaces that can connect (or refuse to connect) a link to a node based on the instructions in the .bdc file. It is also possible to ensure that the link automatically hooks to a particular connection point (and fills one or more Figaro object interfaces), without requiring the user to aim precisely at that point.

In order for the interface filling instructions to have a meaning, it is necessary to put constraints on the possibilities of attaching a given link to given node connection points. These constraints can be declared at the level of the links and/or nodes, in the substructure ACCEPTED_CONNECTION.

At the node level, we define named⁴ **connection points** (not to be confused with Figaro interfaces), which are associated with:

- a graphical position (X and Y coordinates between -100 and 100, which are positions defined with respect to the center (of coordinates (0,0)) of the rectangle inscribed around the icon of the node),
- the list of the links whose hooking on the connection point will be authorized.

Similarly, for each link, we can specify the connection points on which the link can be hooked.

Thus, for example, within the same KB, it will be possible to have a link representing a pipe and going from an "out" connection point of a hydraulic component to an "in" connection point of a hydraulic component, and another link representing an electrical cable and going from an "out" connection point of an electrical component to an "in" connection point of an electrical component. The same result would be obtained by differentiating the names of the connection points according to whether they belong to electrical or hydraulic components; this would make it possible to authorize the connection of the links of the class Figaro, which is the mother of all classes. The first solution is preferable because it avoids the inflation of the vocabulary of the KB, which is a source of errors.

Two different philosophies are possible to geographically place the connection points of a node, depending on whether it is thought that the user will want to aim at specific points or not to hook the links of his model:

- you can give discriminating constraints on the classes of links that can be hooked to each connection point, with rules for filling interfaces adapted to each case (in this case, it will be pointless to aim, and it will be possible to have several superimposed connection points, for example at the point (0, 0)),
- it is also possible to place the connection points far enough apart from each other so that by targeting a specific point the user can decide to connect a link to such or such connection point, which eliminates the need to declare discriminating constraints. In any case, in case of ambiguity (imprecise aim), a dialogue will ask the user at which point he wants to establish the connection.

The first solution is obviously the most satisfactory in the majority of cases; it has the advantage of minimizing the number of connection point names to create and therefore manage later. In particular, it makes it possible not to retouch (except perhaps with regard to the position in X and Y) the connection points created by default when generating the skeleton of the .bdc file from the .fi file.

⁴ The connection points are called "ports" in the VisualFigaro HMI.

However, it may happen that we need to be able to connect standard links (eg electrical wires) on any of the multiple connection points of a component (for example an integrated circuit with many pins); in this case, it is the second solution that must be taken.

4.6 Specify the connection constraints and the rules for filling the interfaces

The connection constraints and the rules for filling the interfaces are grouped in the structures <ACCEPTED_CONNECTION>, which are sub-structures of <CONNECTION_POINT> for the nodes, and of <STARTING_POINT> or <TARGET_POINT> for links.

If it is possible, fill in the structures <ACCEPTED_CONNECTION> only in links. This allows to keep the standard "in" and "out" connection points created by default for all nodes.

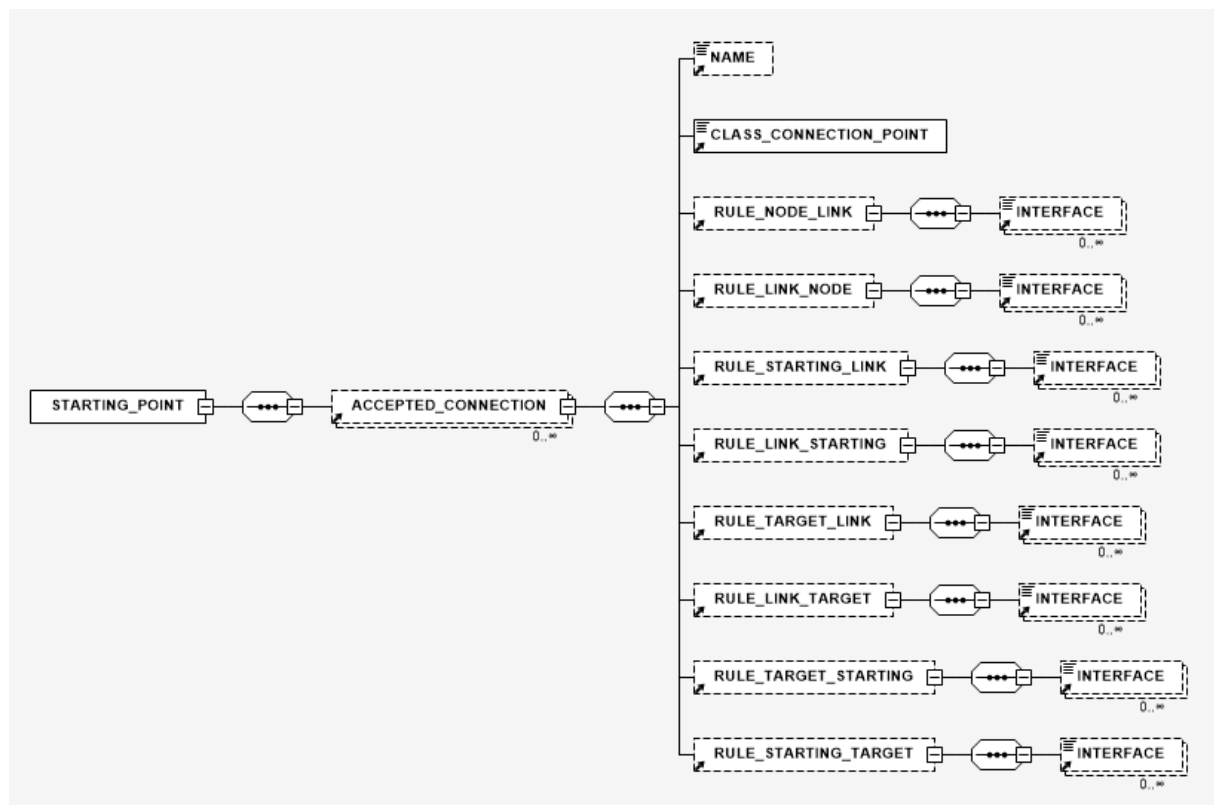
The other benefits of this approach are as follows:

- As in general, there are far fewer links than nodes in a knowledge base, the <ACCEPTED_CONNECTION> information, which is the most complex of a .bdc file, are grouped into a small number of structures, which facilitates the overview.
- the interface filling rules for putting the object from which a link starts into an interface of the object on which the link arrives (or conversely) can only be declared **in the links**.

In fact, the main advantage of using <ACCEPTED_CONNECTION> structures in nodes is to factorize rules that would be repeated in several different links. But this situation seems rather hypothetical.

In a structure <ACCEPTED_CONNECTION>, **only** the field <CLASS_CONNECTION_POINT> is obligatory. The class FIGARO can be specified not to be restrictive.

The interface filling rules all have names of the type RULE_XX_YY and they are associated with one or more interfaces I1 to In.



They have the following meaning: fill the interfaces I1 to In of XX by YY.

Example1: The following extract means that the starting point of the link to which this snippet belongs can be connected to the "output" connection point of the "node" class (and its subclasses).

Such a connection causes the "start" interface of the link itself to be filled by the starting node.

```

<STARTING_POINT>
  <ACCEPTED_CONNECTION>
    <NAME>output</NAME>
    <CLASS_CONNECTION_POINT>node</CLASS_CONNECTION_POINT>
    <RULE_LINK_STARTING>
      <INTERFACE>start</INTERFACE>
    </RULE_LINK_STARTING>
  </ACCEPTED_CONNECTION>
</STARTING_POINT>

```

Example2: The following extract means that the target point of the link to which this snippet belongs can be connected to the "input" connection point of any class (use of keyword FIGARO).

Such a connection causes the "start" interface of the link itself to be filled by the starting node.

```

<TARGET_POINT>
  <ACCEPTED_CONNECTION>
    <NAME>input</NAME>
    <CLASS_CONNECTION_POINT>FIGARO</CLASS_CONNECTION_POINT>
    <RULE_LINK_STARTING>
      <INTERFACE>start</INTERFACE>
    </RULE_LINK_STARTING>
  </ACCEPTED_CONNECTION>
</TARGET_POINT>

```

Example3: This more complex example shows the possibility of filling multiple interfaces, both in the end objects of a link and in the link itself. The following excerpt means that the starting point of the link to which this snippet belongs can be connected to the "output" connection point of the "node" class.

Such a connection causes the interfaces "edges" and "down_edges" of the starting object of the link (thus, a node) to be filled by the link itself, and of the interface "start" of the link by the starting node.

```

<STARTING_POINT>
  <ACCEPTED_CONNECTION>
    <NAME>output</NAME>
    <CLASS_CONNECTION_POINT>node</CLASS_CONNECTION_POINT>
    <RULE_STARTING_LINK>
      <INTERFACE>edges</INTERFACE>
      <INTERFACE>down_edges</INTERFACE>
    </RULE_STARTING_LINK>
    <RULE_LINK_STARTING>
      <INTERFACE>start</INTERFACE>
    </RULE_LINK_STARTING>
  </ACCEPTED_CONNECTION>
</STARTING_POINT>

```

Note: When an <ACCEPTED_CONNECTION> structure contains several interface filling rules, these rules must be in the following order according to the schema defining the structure of the .bdc file, otherwise the KB cannot be loaded into KB3.

- | | |
|-------------------------|-----------------------------|
| 1. RULE_NODE_LINK | (applicable only in a node) |
| 2. RULE_LINK_NODE | (applicable only in a node) |
| 3. RULE_STARTING_LINK | (applicable only in a link) |
| 4. RULE_LINK_STARTING | (applicable only in a link) |
| 5. RULE_TARGET_LINK | (applicable only in a link) |
| 6. RULE_LINK_TARGET | (applicable only in a link) |
| 7. RULE_TARGET_STARTING | (applicable only in a link) |
| 8. RULE_STARTING_TARGET | (applicable only in a link) |

Example 4: This example is taken from a knowledge base containing a class component provided with a connection point "output" and a connection point "input". All subclasses of component inherit from these connection points, with the exception of the toggle class, which inherits from the "output" point, but has,

instead of the "input" connection point, two points called "normal_input" and "backup_input". We need a single type of link, which will fill, **depending on the destination connection point**, either the "normal" interface or the "backup" interface, and in any case the "upstream" interface of its target node.

```

<LINK>
  <STARTING_POINT>
    <ACCEPTED_CONNECTION>
      <NAME>output</NAME>
      <CLASS_CONNECTION_POINT>component</CLASS_CONNECTION_POINT>
    </ACCEPTED_CONNECTION>
  </STARTING_POINT>
  <TARGET_POINT>
    <ACCEPTED_CONNECTION>
      <NAME>input</NAME>
      <CLASS_CONNECTION_POINT>component</CLASS_CONNECTION_POINT>
      <RULE_TARGET_STARTING>
        <INTERFACE>upstream</INTERFACE>
      </RULE_TARGET_STARTING>
    </ACCEPTED_CONNECTION>
    <ACCEPTED_CONNECTION>
      <NAME>normal_input</NAME>
      <CLASS_CONNECTION_POINT>toggle</CLASS_CONNECTION_POINT>
      <RULE_TARGET_STARTING>
        <INTERFACE>normal</INTERFACE>
        <INTERFACE>upstream</INTERFACE>
      </RULE_TARGET_STARTING>
    </ACCEPTED_CONNECTION>
    <ACCEPTED_CONNECTION>
      <NAME>backup_input</NAME>
      <CLASS_CONNECTION_POINT>toggle</CLASS_CONNECTION_POINT>
      <RULE_TARGET_STARTING>
        <INTERFACE>backup</INTERFACE>
        <INTERFACE>upstream</INTERFACE>
      </RULE_TARGET_STARTING>
    </ACCEPTED_CONNECTION>
  </TARGET_POINT>
</LINK>

```

4.7 Specify algorithms to build deduced objects

It is sometimes useful to create additional Figaro objects deduced (automatically) from those entered by the user, before starting treatments. A typical example is the creation of macro-components grouping objects placed in series in the graph corresponding to the topology of a system (like in the Skelbo open source KB). Using macro-components can be useful for the production of fault trees with a more readable structure, or for reducing the combinatorial explosion of treatments. The deduced objects are built by algorithms starting from the Figaro objects manually input by the user.

These algorithms are defined in a language that is an extension of the Figaro language, allowing special actions: creation, deletion or modification of Figaro objects or object fields such as interfaces, constants and attributes. This language is the subject of specific documentation. Deduced objects algorithms are implemented in sub-structures of the structure <DO_ALGORITHM> of the .bdc file.

Note: Deduced objects are not a necessity. Only complex knowledge bases make use of this possibility.

4.8 Choose or create icons and link styles

Once you reach this point, the .bdc file is sufficient to quickly enter systems and start different types of processing. Created objects all look the same, but they are named after their type name followed by a number, which is enough to distinguish them. However, it quickly becomes useful for the readability of models, to particularize the icons and styles of links associated with different classes.

4.8.1 Icons

Use the HMI to choose standard icons (put in the icons directory when creating the knowledge base) or to retrieve others (which in this case will be copied to the icons directory) from other directories.

Note: In addition to the icons corresponding to the objects defined in the KB, it is necessary to always have an icon named "MCG" (thus the two files MCG.ico and MCG.sym) which will be used to represent the graphical macro-components.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 17/23
---------------------------	--	------------------------------

4.8.2 Links

Use the HMI to choose standard icons (put in the icons directory during the creation of the knowledge base) or to retrieve others (which in this case will be copied to the icons directory) from other directories, and edit the link characteristics accessible by the "Edit Default GV" button.

Important note: You have to reload the knowledge base in KB3 to take into account icon changes. On the other hand, there is no need to re-enter the systems.

4.9 Exploit inheritance in the .bdc file

So far, we have only used inheritance in the Figaro language file. It is possible to do it also in the .bdc file. In the latter, inheritance links must be explicit, and must be **a subset of existing inheritance links in the .fi file**. You can skip steps in the hierarchy because you do not have to declare nonterminal types in the .bdc.

For example, if the class hierarchy is the following in the .fi file:

```
-FIGARO
  |-node
  |   |-source
  |   |-target
  |-link
  |   |-mono_dir_link
  |   |-bi_dir_link
  |-fail_counter
```

It will be possible to specify that `source` and `target` inherit some of their characteristics (graphic variants or connection points) from `node`.

There is no inheritance concept to "factorize" connection points for links; on the other hand, we can factor graphic variants.

It is essential to use inheritance to avoid repetitions that ultimately generate errors because the non-use of inheritance enormously inflates the volume of information to manage.

It seems that the introduction of inheritance is optimal at this point of development of the KB. When it is still very unstable because it is being debugged, it seems difficult to do so, because inheritance changes in the .fi file would require changes in the .bdc file each time.

Important Note: Since 2018, it is possible to use inheritance in a classic way (like in the .fi file) in the .bdc file. This new capability reduces the size of the .bdc file, often by a factor of 2 or 3. To switch to this mode, simply declare the AUTOH attribute and set it to TRUE in the first tag:

```
<KB xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="bdceng.xsd" AUTOH="TRUE">
```

and the .bdc file will be correctly interpreted by KB3.

This option is not available in VisualFigaro. It is only accessible with the new editor FigaroIDE of Aristè company, which has the advantage of automating the transition from mode AUTOH="FALSE" to mode AUTOH="TRUE".

4.10 Add display modes and graphic variants

This point is the one that will allow to have a really user-friendly application, even spectacular, with changes in appearance of the graphic model during an interactive simulation.

You must start by declaring the list of names of all display modes that you will need. They appear as follows in the .bdc file:

```
<DISPLAY_MODE>Capacities</DISPLAY_MODE>
<DISPLAY_MODE>Failed components</DISPLAY_MODE>
```

The HMI then makes it possible to choose among them those in which a graphic variant applies.

A graphic variant allows you to:

- Change the color of an icon

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 18/23
---------------------------------------	--	------------------------------

- Make the icon blink
- Change the icon (you must have another .sym file that you associate with the graphic variant)
- Write a text near the icon (TOP, BOTTOM, LEFT or RIGHT). This allows for example to display the value of a variable by designating it as %s within a character string defining a "format". Here is for example how to display in all cases (condition equal to TRUE) a text containing variables and concatenated fixed character strings, to display a first variable, numeric and a second of type enumerated which can take the values 'watts' and 'megawatts':

```

<GRAPHIC_VARIANT>
  <NAME>VG2</NAME>
  <DISPLAY_MODE>Capacities</DISPLAY_MODE>
  <CONDITION>TRUE</CONDITION>
  <TEXT>
    <EXPRESSION>
      <FORMAT>Power = %s %s</FORMAT>
      <PARAMETERS>capacity</PARAMETERS>
      <PARAMETERS>unit</PARAMETERS>
    </EXPRESSION>
    <TEXT_POSITION>TOP</TEXT_POSITION>
  </TEXT>
</GRAPHIC_VARIANT>

```

This will write for example: Power = 15.5 megawatts

Each graphic variant is conditioned both by the visualization mode chosen by the user of KB3 and by a Boolean condition defined in Figaro language. If several graphic variants are applicable in a given situation, only the last one is applied (the effects of the variants are not cumulative).

It is recommended to use inheritance as much as possible for graphic variants.

4.11 Add display layers⁵

In order to be able to quickly switch between different displays, more or less refined, of a complex diagram, you can place the objects into different layers, which you choose to display or not. You can also manage the order of layers, to take into account situations where drawing elements overlap.

Each layer can contain a background image (a .bmp format file, excluding any other format) and a list of objects. Some objects can belong to multiple layers. The user of KB3 can define in each study his/her own layers, but to facilitate the work it is possible to declare in the knowledge base a list of layers, with the classes (and those which inherit from them) to be displayed on these layers. Thus, when creating a new object in KB3, it will automatically be associated with certain layers, depending on the information defined in the KB. The classes declared in the <LAYER> structures must be present elsewhere in the .bdc file.

Here is an example of definition of two layers, one containing a background image as well as the electrical and hydraulic components, and the other containing the control of a system (the layers are to be declared just after the list of the display modes in the .bdc file):

```

<LAYER>
  <NAME>CP</NOM>
  <DESCRIPTION>Main Components</DESCRIPTION>
  <FILE>background_image.bmp</FILE>
  <CLASS>electric_cpt</CLASS>
  <CLASS>hydraulic_cpt</CLASS>
</LAYER>
<LAYER>
  <NAME>Control</NOM>
  <DESCRIPTION>Control components</DESCRIPTION>
  <CLASS>control</CLASS>
</LAYER>

```

4.12 Lock the application

It is time to go back to the .fi file by adding facets EDITION allowing to:

- hide from the final user the fields which he/she will not need (EDITION NOT VISIBLE),

⁵ From KB3 v1.3.2.4.

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 19/23
---------------------------	--	------------------------------

- prevent any modification of field which could introduce inconsistencies in a model (EDITION NOT MODIFIABLE). This is particularly the case of interfaces that are filled automatically by the links layout. It is **very important** to declare this kind of interface as NOT MODIFIABLE, because besides the fact that it will avoid errors due to bad handling by the user, it will have the effect of correctly filling the interfaces of the pasted objects after a copy-and-paste of a group of objects containing links.

Optionally, it is also desirable to add CARDINAL facets for the interfaces for which it is not yet done, so as to control the number of objects that will be declared there.

4.13 Write help for the final user

Two types of help are desirable: offline help and online help.

In any case, it is desirable to offer the user a document independent of KB3, but presented in such a way that one can quickly find the definitions of the classes, with their fields, etc. A file containing a skeleton of help document is automatically copied to the knowledge base directory when it is created. Another possibility is to create an Excel file in which each sheet is associated with a class of the KB, and a "summary" sheet contains the hierarchy of classes.

For the online aspect, it is necessary to inform the field DESCRIPTION associated with each class in the .bdc file, because this field appears to the right of the class icon in the palette. In the same way, it is possible to fill in the .fi file the fields LABEL associated with the various characteristics of the classes: interfaces, constants, attributes, effects, failures. In fact, these labels, besides the fact that they serve to give readable names to the gates of fault trees generated by KB3, are displayed by the GUI of KB3 when clicking on the "+" associated with a name of object in the palette. These LABEL fields benefit from inheritance just like the constants or variables they describe.

5 Modification of an existing KB

5.1 General principles

The main risk when taking over an existing KB is to get lost in the details and to introduce errors because of a lack of overview. That is why we recommend, for doing fairly deep changes in a KB, to start by deleting the .bdc file so as to re-generate a skeleton of .bdc file which will be reduced to a minimum, and to resume the steps defined in this document, without skipping any. It is not forbidden, of course, to save time at each step by using copy and paste from the old version of the .bdc file (this is easily done using QXmlEdit).

Note : version management of a KB. In the case where you do small evolutions, it is convenient to keep the same name of KB (this name is declared in the .bdc file), and to announce the version number in the description. The name of the KB and its description are displayed in KB3; so we can not be wrong on the version of KB that we use. On the other hand, for significant evolutions, it is better to include the version number in the name of the KB. In any case, it is recommended to keep the same names for the *files*.

5.2 How to reload the studies entered with a previous version of KB?

Even if you change a knowledge base quite seriously, you would like to be able to reuse the studies entered with a previous version rather than completely re-enter them.

This can be done to manage the most common changes:

- changes of rules and deduced variables only: you must be able to reload a study directly;
- changes of names of essential variables: the initializations of these variables made in the profiles and variants of KB3 will be lost at the loading of the study;
- interface name changes: the contents of these interfaces will be lost when the study is loaded. This can introduce inconsistencies that are relatively difficult to detect in a large study;
- pure and simple deletion of classes: objects of classes that do not exist in the new KB will be destroyed when the study is loaded;
- changes of names of classes visible in the palette. To rename class A to B, you can add the class B with all its definition to the old knowledge base, load the study with this temporary knowledge base and use KB3 to change the class of all class A objects into class B (it is possible in a single operation).

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 20/23
---------------------------------------	--	------------------------------

Information may be lost during this transformation. Then save the study and reopen it with the new knowledge base.

In all cases, KB3 lists in its log file all the information destroyed when loading the study with a new knowledge base. One can therefore by a thorough examination of this file be informed of all the changes that must be made. For very numerous and repetitive changes, it may be worthwhile to write a small script in SQL to be loaded at the launch of KB3 (by the button "apply patch").

Here is an example of such a script, whose purpose is to bring together in one class "tester_link" objects that belonged to two different classes (hydrau_test_link and elec_test_link). So it is just a question of changing the class name of the old links. Such a script must be in a .kbp extension file.

```
<?xml version="1.0" encoding="utf-8"?>
<DatabaseManagement xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="KB3DatabaseManager.xsd">
  <VersionUpdate fromVersion="24" toVersion="24">
    <Comment>modification of KB classes names</Comment>

    <Command>update STRU_FIG set NOM="tester_link" where
      NOM="hydrau_test_link";</Command>
    <Command>update STRU_FIG set TYPE_NAME="tester_link" where
      TYPE_NAME="hydrau_test_link";</Command>

    <Command>update STRU_FIG set NOM="tester_link" where
      NOM="elec_test_link";</Command>
    <Command>update STRU_FIG set TYPE_NAME="tester_link" where
      TYPE_NAME="elec_test_link";</Command>

  </VersionUpdate>
</DatabaseManagement>
```

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 21/23
---	--	------------------------------

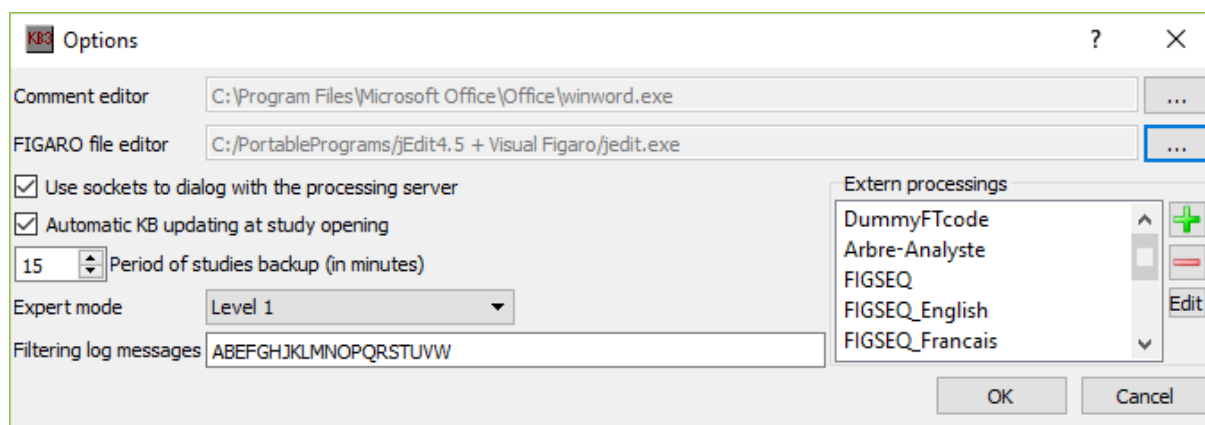
6 Conclusion

The advice we have given in this document should be refined and completed to adapt to:

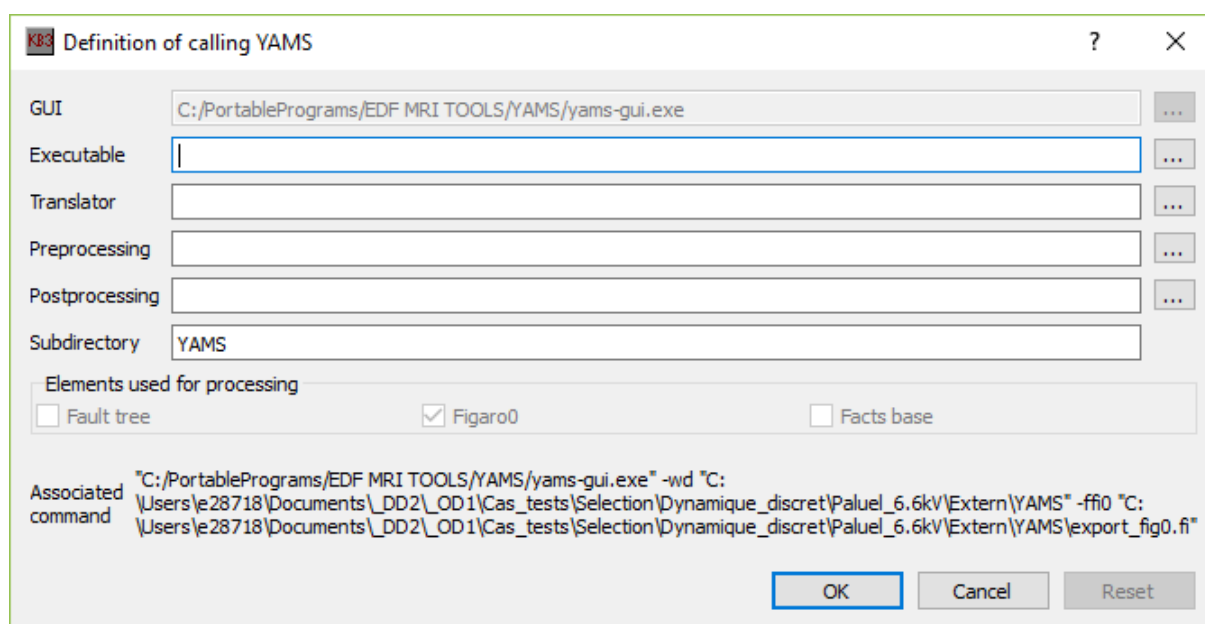
- the evolution of KB3 and the VisualFigaro editor,
- the accumulation of new experiences,
- the appearance of new tools on the market.

7 Appendix A: Passing parameters when KB3 launches an external code

KB3 passes parameters to external codes that it launches. These parameters can be specified in the options panel. You must first select a code in the following window (or create it by clicking on the "+" button if it is not in the list):



... then click on the "Edit" button to access the following panel (here for YAMS):



In this example, you can start YAMS by having it load directly the Figaro 0 file that has just been generated by KB3 (one can do the same thing for Figseq).

Depending on the information chosen in the panel, KB3 displays at the bottom the command that it will launch when the user chooses the processing associated with this code name (here, YAMS). This makes it possible to know how the program that will be launched will have to interpret the arguments that will be passed on to it.

If the launched external code is a .bat file, this file can access the parameters by %1, %2 etc. In the example above, %1 is "-wd", %2 is the working directory, %3 is "-ffi0" and %4 is the full name of the Figaro 0 file YAMS will load.

Here is an example: the launch of the FT2BDMP translator which transforms a fault tree generated by KB3 (with some peculiarities allowing to code the presence of triggers) into a BDMP written in Figaro 0.

It is a .bat file of one line containing the following command:

EDF R&D MRI Department	Knowledge Base Development Manual for KB3 v3 Software	HT-52/03/042/J Page 23/23
---------------------------------------	--	------------------------------

"C:\add2bdmp\add2bdmp.exe" "C:\add2bdmp\BDMP_En.fi" "C:\add2bdmp\OpaleEn.xml" %4 -o
%2\BDMPfig0.fi

This .bat file launches the add2bdmp.exe program with as arguments (in order):

- A file containing the knowledge base BDMP_En.fi
- The OpaleEn.xml parameter file indicating how the triggers of the BDMP are encoded in the names of gates and leaves of the fault tree
- The 4th argument sent by KB3: the full name (with path) of the fault tree file generated by KB3
- The 2nd argument sent by KB3 (name of the directory containing the generated fault tree file) concatenated with "BDMPfig0. fi " to form the output file name of the add2bdmp.exe program.